LLM-Based Code Generation Method for Golang Compiler Testing

Qiuhan Gu
State Key Laboratory for Novel Software Technology, Nanjing University, China
qiuhan.gu@smail.nju.edu.cn

ABSTRACT

Modern optimizing compilers are among the most complex software systems humans build. One way to identify subtle compiler bugs is fuzzing. Both the quantity and the quality of testcases are crucial to the performance of fuzzing. Traditional testcase-generation methods, such as Csmith and YARPGen, have been proven successful at discovering compiler bugs. However, testcases these methods generated have limited coverage and quantity. In this paper, we present a code generation method for compiler testing based on LLM to maximize the quality and quantity of the generated code. In order to avoid undefined behavior and syntax error in generated testcases, we design a filter strategy to clean the source code, preparing a high-quality dataset for the model training. Besides, we present a seed schedule strategy to improve code generation. We apply the method to test the Golang compiler and the result shows that our pipeline outperforms previous methods both qualitatively and quantitatively. It generates testcases with 3.38% average coverage, compared to testcases with 0.44% average coverage generated by GoFuzz. Besides, it detects only 2.79% of syntax errors and 0% of undefined behavior.

CCS CONCEPTS

• **Software and its engineering → Software testing and debugging**.

KEYWORDS

Large model, Code generation, Compiler testing, Go language

ACM Reference Format:
Qiuhan Gu. 2023. LLM-Based Code Generation Method for Golang Compiler Testing. In *Proceedings of the 31st ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE '23), December 3–9, 2023, San Francisco, CA, USA.* ACM, New York, NY, USA, 3 pages. https://doi.org/10.1145/3611643.3617850

1 INTRODUCTION

Compilers are notoriously hard to test, and modern optimizing compilers tend to contain many subtle bugs, leading to, potentially, the introduction of security vulnerabilities that cannot be detected without knowledge of a compiler flaw [1]. The literature on compiler testing is extensive [2]. One core approach to testing compilers is based on the generation of random programs [3]. Csmith is perhaps the most prominent example of this method. Building a tool such as Csmith is a heroic effort, requiring considerable expertise and development time.

The number of testcases generated by random program generation methods is still limited. Moreover, when using methods like Csmith to generate testcases, it is important to consider the coverage and diversity of the testcases, and further manual filtering and supplementation are necessary to ensure tests are all effective. Also, Csmith is focused on a single, although extremely important, language: C. There is no effective tool for generating efficient Go programs. The language Go, albeit not widely used in the research community, has significant applications in many domains, such as high-performance server development and microservices architecture. Go is primarily fuzzed at the official tool Go Fuzzing [4], which has the same defects as Csmith. As a result, we turn to a large-scale model to address the issue of universality. However, although deep learning is an effective way of fuzzing, experience shows that the effectiveness of testcases generated by large-scale model-based fuzzing is limited in practical applications [5].

In this paper, we present an LLM-based high-quality code generation method for testing the Golang compiler. To summarize, our contributions to this work include:

(1) A LLM-based high-quality code generation method.
(2) Apply the method to the Golang compiler, generating testcases with 3.38% average coverage. It detects only 2.79% of syntax errors and 0% of undefined behavior in the testcases.

2 RELATED WORK AND BACKGROUND

2.1 Undefined Behavior

A significant practical problem with using random testing to find miscompilation bugs is that programming languages are unsafe. For example, when a Go-code program executes an erroneous action such as dividing a floating-point number, the Go implementation does not typically flag the violation by throwing an exception or terminating the program. Rather, the erroneous program may continue to execute, but with a corrupted memory state. There are three main kinds of untrapped errors in Go that are referred to as undefined behaviors (UBs) [6]. (1) The initialization order of global variables in packages; (2) The divisor is zero; (3) Race condition.

2.2 Large Language Model

Recently, approaches using large-scale pretrained language models (LMs) have shown promising results in Program synthesis or code generation [7–13], such as GPT-3 [8], which is a massive pretrained language model with 175 billion parameters based on the

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
ESEC/FSE '23, December 3–9, 2023, San Francisco, CA, USA
© 2023 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 979-8-4007-0327-0/23/12…$15.00
https://doi.org/10.1145/3611643.3617850

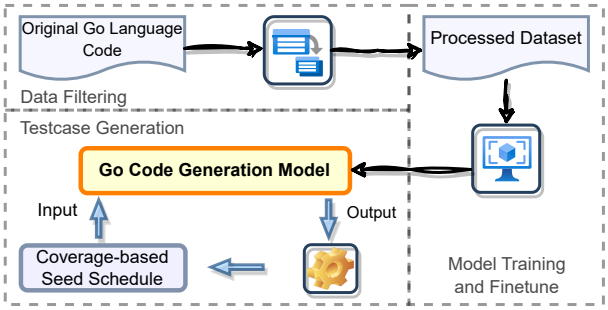

Figure 1: The whole workflow of our experiment. This workflow consists of three parts: (1) data filtering, which involves removing undefined behaviors and syntax errors from the dataset; (2) model training and fine-tuning; (3) a loop including seed schedule, generating a great number of testcases.

Transformer architecture. They can address the challenges of long development cycles, high development complexity, and poor cross-language portability that are associated with traditional compiler development tools. However, they are also difficult to deploy on regular hardware devices due to high parameter count and large size.

CodeT5 [13] is a multi-lingual code-aware language model pre-trained on large-scale source code corpora curated from Github. With a unified encoder-decoder architecture, CodeT5 achieves state-of-the-art performance in a wide range of code intelligence tasks in the CodeXGLUE benchmark [14].

3 CODE GENERATION FOR GO BASED ON LLM

3.1 Training Data Processing

In this design, we process Go language code files from the internet with the help of the syntax analysis tool tree-sitter [15] to obtain Go language code files that meet our requirements.

Firstly, we remove all comments from the code. Then we designed a series of filtering criteria to remove low-quality programs:

(1) Remove files with syntax errors.
(2) Remove files with a character length exceeding 10000.
(3) Remove files with duplicate code[16].
(4) Remove files with an alphanumeric characters ratio below 0.25.
(5) Remove files that may contain undefined behavior.
(6) Remove files that reference the "internal" package.

Thus, we prepare the filtered dataset containing 1839 programs for model training and initial seed choice.

3.2 Maximizing Coverage of Testcases

Many Fuzzing strategies always base on known error reports, vulnerability records, or the boundary conditions of input to choose seed samples. Go Fuzz chooses seeds that lead to the execution of more code paths, thereby increasing the probability of discovering potential issues. However, they are not suitable for our purpose, which is to test the compiler not executable code with parameters, it is important to consider the code coverage of the compiler itself rather than the code coverage of testcases.

In this paper, we propose a code coverage-based seed schedule strategy for the compiler, which includes the following steps:

(1) Use gotests [17] to calculate the coverage of each initial testcase for the Golang compiler and sort them based on coverage. ("Coverage" refers to the measure of how much of the source code of the Golang compiler is exercised by the given testcases.)
(2) Select the testcase with the highest coverage as the seed, randomly remove a function body from it as the input, and then generate several new testcases by combing the input and output.
(3) Add the new testcases to the original queue and resort them based on coverage.
(4) Repeat steps 1-3 until the coverage reaches a stable state.

3.3 Generating Testcases Automatically

We can generate tastcases automatically and infinitely. It's another main difference between our method and other common methods like GoFuzz, Csmith, and YARPGen.

In our experiment, we base our work on the concode task (code generation task) in the CodeT5 model and improve the performance and prediction accuracy of the model through feedback training. Finally, we obtain a model capable of automatically generating the body of Go language functions based on the given context of Go language classes provided in the input. By concatenating the input and output, we obtain a complete testcase. Finally, we omit uncompilable testcases. By continuously selecting a testcase, randomly removing one function body from it, and transferring it into the model, we can obtain different presentations of the same function body, thus obtain an infinite number of testcases. As we show in Figure 1.

3.4 Experiment Results

In our experiment, we finetuned the CodeT5 through 30 epochs, at the learning rate of 10e-5 and warmup steps of 1000, to get the final model. Then we implemented our method on the Golang compiler, looping 1000 times to generate 1157 testcases (cost about 100h).

We detected only 2.79% of syntax errors and 0% of undefined behavior in testcases our tool generated. The average coverage of the generated code is 3.38%, compared to testcases with 0.44% average coverage generated by Go Fuzzing [18], demonstrating our method does particularly well in this aspect. Our dataset and tool are publicly available at [19].

4 CONCLUSION

We present an LLM-based code generation method to test modern compilers efficiently. With the help of a large language model, we achieve the universality and portability of the tool, and increase the quantity of generated testcases, significantly reducing manual effort. We present a code-filtering strategy to avoid syntax errors and undefined behavior in generated testcases. Finally, we use a coverage-based seed schedule to improve the performance of testcases. As a result, our method outperforms previous testing methods both qualitatively and quantitatively.

5 ACKNOWLEDGMENTS

This work was supported by the National Natural Science Foundation of China under Grant No. 62232001 and No. 62202220.

REFERENCES

[1] Michaël Marcozzi, Qiyi Tang, Alastair F. Donaldson, and Cristian Cadar. Compiler fuzzing: How much does it matter? (OOPSLA), 2019.

[2] Junjie Chen, Jibesh Patra, Michael Pradel, Yingfei Xiong, Hongyu Zhang, Dan Hao, and Lu Zhang. A survey of compiler testing. *ACM Computing Surveys (CSUR)*, 53(1):1–36, 2020.

[3] William McKeeman. Differential testing for software. *Digital Technical Journal of Digital Equipment Corporation*, 10(1):100–107, 1998.

[4] Go.dev - fuzzing. https://go.dev/security/fuzz/. Accessed: August 30, 2023.

[5] Eirini Kalliamvakou, Georgios Gousios, Kelly Blincoe, Leif Singer, Daniel M. German, and Daniela Damian. The perils and pitfalls of mining github. *Proceedings of the 13th International Conference on Mining Software Repositories*, 2016.

[6] Go. https://go.dev/ref/spec. Accessed: August 30, 2023.

[7] Pengcheng Yin, Graham Neubig, and Prasanna Parthasarathi. Codet5: Synthesizing robust testers for language primitives from semi-structured natural language instructions. In *Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics (ACL)*, pages 1843–1857, 2021.

[8] Alec Radford, Jeffrey Wu, Rewon Child, David Luan, Dario Amodei, and Ilya Sutskever. Language models are unsupervised multitask learners. *OpenAI Blog*, 2019.

[9] Jared Kaplan, Sam McCandlish, Tom Henighan, Tom B. Brown, Benjamin Chess, Rewon Child, Scott Gray, Alec Radford, Jeffrey Wu, and Dario Amodei. Scaling laws for neural language models. *arXiv preprint arXiv:2001.08361*, 2020.

[10] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. Improving language understanding by generative pre-training. *arXiv preprint arXiv:1810.04805*, 2018.

[11] Matej Balog, Alexander L. Gaunt, Marc Brockschmidt, Sebastian Nowozin, and Daniel Tarlow. Deepcoder: Learning to write programs. In *International Conference on Learning Representations*, 2017.

[12] Richard Shin, Dawn Song, and Xinyun Chen. Alphacoder: Teaching an ai to write programs. In *Advances in Neural Information Processing Systems*, 2020.

[13] Yue Wang, Weishi Wang, Shafiq Joty, and Steven C.H. Hoi. Codet5: Identifier-aware unified pre-trained encoder-decoder models for code understanding and generation. *arXiv preprint arXiv:2109.00859*, 2021. Accepted to EMNLP 2021. 13 pages.

[14] Shuai Lu, Daya Guo, Shuo Ren, Junjie Huang, Alexey Svyatkovskiy, Ambrosio Blanco, Colin Clement, Dawn Drain, Daxin Jiang, Duyu Tang, et al. Codexglue: A machine learning benchmark dataset for code understanding and generation. *arXiv preprint arXiv:2102.04664*, 2021.

[15] Tree-sitter. https://github.com/tree-sitter/tree-sitter. Accessed: August 30, 2023.

[16] Miltiadis Allamanis. The adverse effects of code duplication in machine learning models of code, 2019.

[17] Gotests. https://github.com/cweill/gotests. Accessed: August 30, 2023.

[18] Gofuzzurl. https://github.com/dvyukov/go-fuzz. Accessed: August 30, 2023.

[19] Experiment code: Llm-based-code-generation-method-for-golang-compiler-testing, including our dataset, finutuned model and scripts. https://github.com/GuQiuhan/LLM-Based-Code-Generation-Method-for-Golang-Compiler-Testing. Accessed: August 30, 2023.

Received 2023-06-05; accepted 2023-08-11